

Programa: Registro de Empleados

Una Implementación CRUD con Arquitectura Limpia y Blazor Server

Este proyecto es una aplicación web minimalista desarrollada en C# que implementa las operaciones fundamentales de mantenimiento de datos (CRUD: Crear, Leer, Actualizar y Eliminar) sobre una entidad Empleado. Su diseño se distingue no solo por su funcionalidad, sino por su adherencia estricta a los principios de la **Arquitectura Limpia (Clean Architecture)** y los principios **SOLID**, demostrando cómo separar correctamente las responsabilidades en un sistema moderno.

A continuación, se detalla cómo el sistema aborda cada aspecto clave:

1. Arquitectura Limpia y Separación de Capas

El código está organizado en una estructura de capas bien definida que elimina las dependencias cruzadas y garantiza que el núcleo del negocio sea independiente de la interfaz de usuario o de la base de datos:

- Capa de Dominio:** Contiene la entidad Empleado, que es pura lógica de negocio sin dependencias externas.
- Capa de Aplicación:** Orquesta las reglas de negocio y la validación de datos, actuando como intermediaria entre la UI y el repositorio.
- Capa de Infraestructura:** Implementa los detalles técnicos (acceso a SQLite) sin afectar a las capas superiores.
- Capa Web:** Se encarga exclusivamente de la presentación y la interacción con el usuario.
- Beneficio:** Esto permite cambiar la tecnología de la base de datos o la interfaz de usuario sin tocar la lógica de negocio, facilitando enormemente el mantenimiento y la escalabilidad.

2. Implementación de Principios SOLID

- Single Responsibility (SRP):** Cada clase tiene una única razón para cambiar. Empleado maneja sus propios datos, ServicioEmpleados gestiona las reglas de validación y RepositorioEmpleadosSqlite se ocupa exclusivamente de la persistencia en disco.
- Open/Closed (OCP):** La aplicación es extensible pero no modificable en su núcleo. Gracias a la abstracción, podríamos añadir un nuevo repositorio (ej. PostgreSQL) sin modificar el servicio existente.
- Liskov Substitution (LSP):** Cualquier implementación de IRepositoryEmpleados puede sustituir a la concreta sin alterar el comportamiento correcto del sistema.
- Interface Segregation (ISP):** La interfaz IRepositoryEmpleados es pequeña y específica, evitando que las clases implementen métodos que no utilizan.
- Dependency Inversion (DIP):** Las capas de alto nivel (Aplicación) no dependen de los módulos de bajo nivel (Infraestructura), sino de abstracciones (interfaces).

3. Inyección de Dependencias (DI) y Gestión de Ciclos de Vida

El sistema utiliza el contenedor de dependencias nativo de ASP.NET Core para gestionar la creación y el alcance de los objetos:

- Singleton para el Repositorio:** IRepositoryEmpleados se registra como Singleton. Dado que el repositorio abre y cierra su conexión a SQLite de forma transaccional en cada método, no hay riesgo de estados compartidos. Esto evita la sobrecarga de inicializar la base de datos en cada solicitud.
- Scoped para el Servicio:** ServicioEmpleados se registra como Scoped. Esto asegura que cada sesión de usuario (o circuito de Blazor) tenga su propia instancia del servicio, manteniendo un estado limpio y seguro para cada interacción.

4. Integración con Blazor Server

La interfaz de usuario se construye utilizando **Blazor Server**, que permite escribir lógica de UI en C# en lugar de JavaScript. Al ser Server-Side, la aplicación mantiene un canal de comunicación en tiempo real (SignalR) con el cliente, lo que permite actualizaciones dinámicas y validaciones de formularios fluidas sin necesidad de recargar la página.

5. Persistencia con SQLite

Se ha elegido **SQLite** como motor de base de datos por ser un sistema autocontenido y sin configuración de servidor. La base de datos se almacena como un único archivo (`empleados.db`), lo que hace que la aplicación sea extremadamente portable y fácil de desplegar, ideal para proyectos de pequeña escala o prototipos.